

Zecpath AI Job Portal

Day 20 — Final ATS Evaluation Report

ATS AI Module — Production Readiness Review

Repository: `jeffyjohnbinu/ai-recruitment-system`
New module under review: `ats_production_system` (Day 20)

Verdict: FUNCTIONALLY PRODUCTION-READY (ATS AI service scope only — see Section 5)

Prepared for: Zecpath AI Job Portal engineering review

Scope: ATS AI microservice only (per the PRD's five-service architecture)

Modules reviewed: Days 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

New deliverable: ats_production_system (Day 20)

1. Executive Summary

This review validates the ATS AI module end-to-end rather than module-by-module. The most significant finding is that the Day 17 test harness never actually exercised the real Day 12-14 semantic matching and scoring engines: its `pipeline_adapter.py` imports functions that do not exist in those modules

(`ats_scoring_engine.scorer.score_candidate`, for example, has never been defined), so every prior test run silently fell through to a basic keyword-overlap fallback. The previously reported 87.5% accuracy figure describes that fallback, not the production pipeline.

A second, related issue: even with correct imports, the real engines' output field names do not match what the scoring engine's loaders expect (Semantic Matching emits `overall_score` where the loader wants `similarity_score`; Education/Certification extraction produces no alignment score against a job at all). This is why the components were designed to gracefully report 'unavailable' rather than crash -- but 'unavailable' had, in practice, become the default outcome for two of four scoring components.

This report documents the fix: a new orchestration module, `ats_production_system`, that wires the real engines together with verified import paths and a small field-mapping adapter layer. It was validated against a 6-candidate, 2-job-posting demo dataset and a 12-test automated integration suite, both included as Day 20 deliverables. With this fix in place, the ATS AI module is functionally sound end-to-end. It is not yet production-deployable in the full infrastructure sense -- there is no persistent data store, authentication, or observability layer -- and Section 5 lists those gaps explicitly rather than implying completeness.

2. Architecture

The ATS AI pipeline processes one resume against one job posting through eleven stages:

- Day 5 — `ResumeExtractionEngine`: extracts and cleans raw resume text (PDF/DOCX).
- Day 18 — `PerformanceOptimizationEngine.repair_noisy_text`: best-effort noise cleanup (applied, non-blocking).
- Day 8 — `SectionClassifierEngine`: segments cleaned text into labeled sections.
- Day 9 — `SkillExtractionEngine`: extracts and normalizes candidate skills.
- Day 10 — `ExperienceParsingEngine`: builds a work-history timeline and scores role relevance against the target job.
- Day 11 — `EducationCertificationExtractor`: extracts degrees and certifications.
- Day 6 — `JDParsingEngine` (job side): extracts required/preferred skills, experience and education requirements.
- Day 12 — `SemanticMatchingEngine`: embedding-based similarity across skills/experience/projects sections.
- Day 20 — `component_adapters.py`: maps each engine's real output fields onto what the scoring engine expects (the fix).
- Day 13 — `ATSScoringEngine`: combines all four components into one explainable, weighted final score.
- Day 14 — `CandidateRankingEngine`: sorts and zones a candidate pool (Shortlist / Review / Auto-Reject) per job.
- Day 15 — `FairnessBiasEngine`: normalizes scores across the pool and audits for bias when demographic labels are supplied.

Every stage is wrapped so a missing or failing upstream package degrades only that one component (logged as a warning, with weights renormalized across the remaining components) rather than crashing the run. This is the same two-pass fallback discipline used since Day 5's PDF/DOCX reader fallback.

3. The Finding, in Detail

3.1 Broken import paths (Day 17 harness)

ats_system_testing/pipeline_adapter.py attempts these import paths, in order, before falling back to keyword overlap: ats_scoring_engine.scorer.score_candidate, ats_scoring_engine.score_candidate, semantic_matching_engine.matcher.match, and candidate_ranking_engine.ranker.score_candidate. None of these exist in the real Day 12-14 codebase -- the actual entry points are ATSScoringEngine.compute_score, SemanticMatchingEngine.match, and CandidateRankingEngine.rank_job, all class methods rather than the module-level functions the adapter guesses at. Because the adapter catches ImportError and continues silently, this was invisible in every prior test run: the suite still passed, using the fallback.

3.2 Field-name mismatches (real engines)

Once the real engines are imported correctly, their output shapes still don't line up with ats_scoring_engine/loaders.py's expected field names:

- Semantic Matching Engine returns overall_score; the loader looks for similarity_score.
- Experience Relevance Engine returns overall_relevance_score; the loader looks for relevance_score.
- Education/Certification Extractor returns raw degree/certification lists with no alignment score or meets_minimum_education / field_match booleans against a target job at all.
- Skill Extraction Engine returns a flat skill list, not the matched_skills / required_skills pair the loader's ratio fallback needs.

component_adapters.py (Day 20) resolves each of these explicitly -- see the module's docstring and inline comments for the exact mapping and the reasoning behind each derived signal (e.g. how meets_minimum_education is ranked, or how matched_skills is computed from the candidate/job skill sets).

4. Live Demo Results

A 6-resume demo dataset (varied seniority and domain fit) was scored against two distinct job postings to confirm the pipeline discriminates meaningfully rather than returning a flat or arbitrary score.

Job: Senior Backend Engineer

Candidate	Score	Zone	Status	Profile
priya_sharma	0.66	Review	success	Strong fit
daniel_okafor	0.41	Auto-Reject	success	Junior, partial fit
thomas_becker	0.18	Auto-Reject	success	Wrong domain (data analyst)
arjun_nair	0.16	Auto-Reject	success	Wrong domain (data analyst)
megan_ho	0.13	Auto-Reject	success	No relevant fit (marketing)

Candidate	Score	Zone	Status	Profile
sofia_rossi	0.13	Auto-Reject	success	No relevant fit (ops/Excel)

Job: Junior Data Analyst

Candidate	Score	Zone	Status	Profile
arjun_nair	0.59	Review	success	Strong fit
thomas_becker	0.57	Review	success	Overqualified, still relevant
priya_sharma	0.23	Auto-Reject	success	Wrong domain (backend eng.)
daniel_okafor	0.22	Auto-Reject	success	Wrong domain (backend eng.)
megan_ho	0.22	Auto-Reject	success	Partial fit (Excel only)
sofia_rossi	0.22	Auto-Reject	success	Partial fit (Excel only)

The same candidate pool produces a different top candidate for each posting (priya_sharma for backend, arjun_nair for the analyst role), confirming the ranking is job-specific rather than a fixed candidate-quality ordering.

4.1 Component-level breakdown (example: priya_sharma vs. Senior Backend Engineer)

Component	Available	Raw Score	Weight	Contribution
skill_match	Yes	1.00	0.35	0.350
experience_relevance	Yes	0.63	0.25	0.157
education_alignment	Yes	1.00	0.15	0.150
semantic_similarity	Yes	0.00	0.25	0.000

Narrative explanation generated by Day 13's explainability module: "Candidate priya_sharma scored 66% overall against job job_senior_backend (role profile: default), driven primarily by strong skill match (100%), while semantic similarity was comparatively weaker (0%)."

Note on the 0% semantic-similarity component: this sandbox has no outbound network access to Hugging Face, so the Semantic Matching Engine used its offline hashing-vector fallback rather than the sentence-transformers model. The fallback's match-band thresholds were calibrated for the stronger embedder, so semantic scores and match_band should be read as directional in this environment, not as the production-calibrated signal. This is logged automatically on every run via engines_used.semantic_matching.

5. Production Readiness Checklist

Area	Status	Notes
ATS core pipeline (Days 5-15, 18) wired correctly	Ready	Verified with real imports + field-mapping adapters; 12/12 integration tests pass against live engines, not fallbacks.
Explainable scoring	Ready	Every score traces to 4 weighted, named components with a narrative explanation (Day 13).
Candidate ranking & shortlisting	Ready	Day 14 zones (Shortlist/Review/Auto-Reject) confirmed to discriminate correctly across two distinct job postings.
Fairness & bias auditing	Ready	Day 15 pool-level score normalization runs on every batch; four-fifths-rule audit available when demographic labels are supplied out-of-band.
API service	Partial	Core Day 16 endpoints (resumes, job-descriptions, matches, shortlist, health) work end-to-end. Async task polling, PATCH endpoints, and shortlist export are not implemented.
Semantic embeddings	Partial	Falls back to the offline hashing embedder without Hugging Face network access; weaker synonym/paraphrase matching than the sentence-transformers path, and the match-band thresholds are calibrated for the stronger embedder, not the fallback.
Persistent data store	Not ready	API keeps uploads/matches in process memory. Needs a real database + object storage before multi-instance or restart-safe deployment.
Authentication & authorization	Not ready	No auth layer exists on the API. Required before exposing this outside a trusted internal network.
Containerized deployment	Ready	Dockerfile + healthcheck provided; no orchestration (k8s/compose) or CI/CD deploy step yet, only CI test workflow.
Observability	Not ready	Standard logging only; no metrics, tracing, or alerting.
PII / compliance handling	Not ready	No data retention policy, encryption-at-rest, or access controls for candidate PII.
Screening AI / Interview Intelligence / Behavior Analysis	Not ready	Out of scope for this module -- this report covers the ATS AI service only, per the PRD's five-service split.

6. Test Coverage

ats_production_system/tests/test_integration.py (12 tests, all passing) covers: engine availability across all eleven components; job-skill extraction; full four-component wiring for a strong-fit candidate (a direct regression guard against the Section 3 finding); score discrimination between strong and weak candidates; score bounds; a dedicated regression test for the semantic-engine field-mapping fix; batch ranking and zone assignment; pool-level fairness auditing; per-job ranking differences across the same candidate pool; graceful degradation when an engine is deliberately disabled; and correct failure handling for unsupported file types.

This directly addresses the ats_system_testing PERF-001 backlog item, which noted that full benchmark runs needed to happen inside the fully integrated repository -- this suite is that integrated run, using verified imports rather than the harness's guessed ones.

7. Recommendation

The ATS AI module's core logic -- extraction, parsing, matching, scoring, ranking, and fairness auditing -- is sound, explainable, and now verifiably wired end-to-end. It is ready to move from “component library” to “service” for internal / pilot use behind the provided FastAPI service and Docker image.

It is not yet ready for external production deployment. The remaining gaps (persistent storage, authentication, observability, PII/compliance handling, and the two unbuilt PRD microservices) are infrastructure and scope items rather than defects in the ATS logic itself, and are listed in Section 5 so they can be scheduled as their own follow-on work rather than discovered later.

Recommended sign-off position: approve the ATS AI module for pilot/internal deployment, pending the Section 5 “Not ready” items before external production use.

8. Management Review & Sign-off

To be completed following the live demo walkthrough referenced in the Day 20 brief.

Role	Name	Decision (Approve / Approve with conditions / Hold)	Date
Product / Program Owner			
Engineering Lead			
QA / Test Owner			